Object Oriented Programming: Python BootCamp

HIT-emlyon BSc Program
April-May, Harbin | © Imène BRIGUI | Saeed VARASTEH

Lecture 08 : Pandas Library

Pandas is one of the most important libraries of Python. Pandas has data structures for data analysis. The most used of these are **Series** and **DataFrame** data structures. Series is one dimensional, that is, it consists of a column. DataFrame is two-dimensional, i.e. it consists of rows and columns.

Recap:

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and subpackages and sub-subpackages, and so on.

To install Pandas library, you can use "pip install pandas".

Pandas is well suited for many different kinds of data:

- Tabular data with heterogeneously-typed columns, as in an SQL table or Excel spreadsheet
- Ordered and unordered (not necessarily fixed-frequency) time series data.
- Arbitrary matrix data with row and column labels
- Any other form of observational / statistical data sets.

In [ ]:
```python
import pandas as pd
```

DataFrame

DataFrame is the most widely used data structure in Python pandas. You can imagine it as a table in a database or a spreadsheet.

Dataframe is a tabular(rows, columns) representation of data. It is a **two-dimensional** data structure with potentially heterogeneous data.

Dataframe is a size-mutable structure that means data can be added or deleted from it, unlike data series, which does not allow operations that change its size.

DataFrame creation

Data is available in various forms and types like CSV or JSON files, SQL table, Python structures like list, dict and so on.

We need to convert all such different data formats into a DataFrame so that we can use pandas libraries to analyze such data efficiently.

To create DataFrame, we can use either the DataFrame constructor or pandas built-in functions. Below are some examples.

DataFrame constructor

`pandas.DataFrame(data=None, index=None, columns=None, dtype=None, copy=False)`

Parameters:

- `data` : It takes input `dict`, `list`, `set`, `ndarray`, `iterable`, or DataFrame. If the input is not provided, then it creates an empty DataFrame. The resultant column order follows the insertion order.
- `index` : (Optional) It takes the list of row index for the DataFrame. The default value is a range of integers 0, 1,...n.
- `columns` : (Optional) It takes the list of columns for the DataFrame. The default value is a range of integers 0, 1,...n.
- `dtype` : (Optional) By default, It infers the data type from the data, but this option applies any specific data type to the whole DataFrame.
- `copy` : (Optional) Copy data from inputs. Boolean, Default False. Only affects DataFrame or 2D array-like inputs

---

Dataframe from dict

When we have data in `dict` or any default data structures in Python, we can convert it into DataFrame using the DataFrame constructor.

To construct a DataFrame from a `dict` object, we can pass it to the DataFrame constructor `pd.DataFrame(dict)`. It creates DataFrame using, where `dict` keys will be column labels, and `dict` values will be the columns' data.

In [ ]:
```python
# Python dict object
student_dict = {'Name':['Joe','Nat'], 'Age':[20,21], 'Marks':[85.10, 77.80]}
student_dict
```

'Name', **'Age'** and **'Marks'** are the keys in the `dict` when you convert they will become the column labels of the DataFrame.

In [ ]:
```python
# Create DataFrame from dict
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# Another example with the indices
df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],'C': [0.496714, -0.138264, 0.647689]},
                  index=['i', 'ii', 'iii'])
df
```

Dataframe from List

The List is a simple data structure in Python that stores the values as a List. The List can have heterogeneous elements, i.e., it can have values of different types. To analyze such a List, we can convert it into the pandas DataFrame. By converting the List into a 2-dimensional structure makes it efficient to process.

DataFrame can be created from List using DataFrame constructor:

Here we create a DataFrame object using a list of heterogeneous data. By default, all list elements are added as a row in the DataFrame. And row index is the range of numbers (starting at 0).

In [ ]:
```python
# Create list
fruits_list = ['Apple', 10, 'Orange', 55.50]
print(fruits_list)
```

In [ ]:
```python
# Create DataFrame from list
fruits_df = pd.DataFrame(fruits_list)
fruits_df
```

Customized column name and index

We can specify column labels into the columns=[col_labels] parameter in the DataFrame constructor.

We can specify row index into the index=[row_index1, row_index2] parameter in the DataFrame constructor. By default, it gives a range of integers as row index i.e. 0, 1, 2, ..., n.

Example:

In [ ]:
```python
# Create list
fruits_list = ['Apple', 'Banana', 'Orange','Mango']

# Create DataFrame from list
fruits_df = pd.DataFrame(fruits_list, columns=['Fruits'], index=['Fruit1', 'Fruit2', 'Fruit3', 'Fruit4'])
fruits_df
```

Multi-dimensional list

In the below example, we have a list that has lists of fruit names and their prices. DataFrame constructor will add both the lists as a separate row in the resulting DataFrame:

In [ ]:
```python
# Create list
fruits_list = [['Apple', 'Banana', 'Orange', 'Mango'],[120, 40, 80, 500]]
```

In [ ]:
```python
# Create DataFrame from list
fruits_df = pd.DataFrame(fruits_list)
fruits_df
```

In [ ]:
```python
# Create DataFrame from list
fruits_df = pd.DataFrame(fruits_list).transpose()
fruits_df
```

Indexing

One improvement over NumPy arrays is labeled indexing. We can select subsets by column, row, or both. Column selection uses the regular python `__getitem__` machinery. Pass in a single column label `'A'` or a list of labels `['A', 'C']` to select subsets of the original `DataFrame`.

In [ ]:
```python
df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],'C': [0.496714, -0.138264, 0.647689]},
                  index=['i', 'ii', 'iii'])
df
```

In [ ]:
```python
# Single column, reduces to a Series
df['A']
```

In [ ]:
```python
cols = ['A', 'C']
df[cols]
```

For row-wise selection, use the special `.loc` accessor.

In [ ]:
```python
df.loc[['i', 'iii']]
```

You can use ranges to select rows or columns.

In [ ]:
```python
df.loc['i':'iii']
```

Notice that the slice is *inclusive* on the right side, unlike your typical slicing of a list. Sometimes, you'd rather slice by *position* instead of label. `.iloc` has you covered:

In [ ]:
```python
df.iloc[[0, 1]]
```

In [ ]:
```python
df.iloc[:2]
```

This follows the usual python slicing rules: closed on the left, open on the right.

As I mentioned, you can slice both rows and columns. Use `.loc` for label or `.iloc` for position indexing.

In [ ]:
```python
df.loc['ii', 'C']
```

Pandas, like NumPy, will reduce dimensions when possible. Select a single column and you get back `Series` (see below). Select a single row and single column, you get a scalar.

You can get pretty fancy:

In [ ]:
```python
df.loc['i':'ii', ['A', 'C']]
```

Summary

- Use `[]` for selecting columns
- Use `.loc[row_lables, column_labels]` for label-based indexing
- Use `.iloc[row_positions, column_positions]` for positional index

I've left out boolean and hierarchical indexing, which we'll see later.

Series

You've already seen some **Series** up above. It's the 1-dimensional analog of the DataFrame. Each column in a **DataFrame** is in some sense a **Series**. You can select a **Series** from a DataFrame in a few ways:

In [ ]:
```python
# __getitem__ like before
df['A']
```

In [ ]:
```python
# .loc, like before
df.loc[:, 'A']
```

In [ ]:
```python
# using `.` attribute lookup
df.A
```

In [ ]:
```python
# Create DataSeries:
s = pd.Series([2, 4, 6, 8, 10])
s
```

Series share many of the same methods as **DataFrame**s.

Note on Index

`Index` are something of a peculiarity to pandas.

First off, they are not the kind of indexes you'll find in SQL, which are used to help the engine speed up certain queries.

In pandas, `Index` are about lables. This helps with selection (like we did above) and automatic alignment when performing operations between two **DataFrames** or **Series**.

R does have row labels, but they're nowhere near as powerful (or complicated) as in pandas. You can access the index of a **DataFrame** or **Series** with the `.index` attribute.

There are special kinds of `Index`es that you'll come across. Some of these are

- **`MultiIndex`** for multidimensional (Hierarchical) labels
- **`DatetimeIndex`** for datetimes
- **`Float64Index`** for floats
- **`CategoricalIndex`** for, you guessed it, **Categoricals**

Dataframe from CSV

In the field of Data Science, **[CSV]** files are used to store large datasets. To efficiently analyze such datasets, we need to convert them into pandas DataFrame.

To create a DataFrame from CSV, we use the `read_csv('file_name')` function that takes the file name as input and returns DataFrame as output.

Example 1

In [ ]:
```python
data1 = pd.read_csv("data/sample.csv")
data1
```

In [ ]:
```python
type(data1)
```

In [ ]:
```python
data1.iloc[:,2]
```

DataFrame metadata

Sometimes we need to get metadata of the DataFrame and not the content inside it. Such metadata information is useful to understand the DataFrame as it gives more details about the DataFrame that we need to process.

In this section, we cover the functions which provide such information of the DataFrame.

`DataFrame.info()` is a function of DataFrame that gives metadata of DataFrame. Which includes,

- Number of rows and its range of index
- Total number of columns
- List of columns
- Count of the total number of non-null values in the column
- Data type of column
- Count of columns in each data type
- Memory usage by the DataFrame

In [ ]:
```python
data1.info()
```

DataFrame statistics

`DataFrame.describe()` is a function that gives mathematical statistics of the data in DataFrame. But, it applies to the columns that contain numeric values.

In our example of data1 DataFrame, it gives descriptive statistics of **'age'** and **'id'** columns only, that includes:

1. **count**: Total number of non-null values in the column
2. **mean**: an average of numbers
3. **std**: a standard deviation value
4. **min**: minimum value
5. **25%**: 25th percentile
6. **50%**: 50th percentile
7. **75%**: 75th percentile
8. **max**: maximum value

> **Note:** Output of `DataFrame.describe()` function varies depending on the input DataFrame.

In [ ]:
```python
data1.describe()
```

DataFrame attributes

DataFrame has provided many built-in attributes. Attributes do not modify the underlying data, unlike functions, but it is used to get more details about the DataFrame.

Following are majorly used attributes of the DataFrame:

Attribute	Description
`DataFrame.index`	**It gives the Range of the row index**
`DataFrame.columns`	**It gives a list of column labels**
`DataFrame.dtypes`	**It gives column names and their data type**
`DataFrame.values`	**It gives all the rows in DataFrame**
`DataFrame.empty`	**It is used to check if the DataFrame is empty**
`DataFrame.size`	**It gives a total number of values in DataFrame**
`DataFrame.shape`	**It a number of rows and columns in DataFrame**

In [ ]:
```python
# Create DataFrame from dict
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
print("DataFrame Index : ", student_df.index, "\n")
print("DataFrame Columns : ", student_df.columns, "\n")

print("DataFrame Column types : \n", student_df.dtypes, "\n")

print("DataFrame is empty? : ", student_df.empty, "\n")

print("DataFrame Shape : ", student_df.shape, "\n")
print("DataFrame Size : ", student_df.size, "\n")

print("DataFrame Values : \n", student_df.values, "\n")
```

DataFrame selection

While dealing with the vast data in DataFrame, a data analyst always needs to select a particular row or column for the analysis. In such cases, functions that can choose a set of rows or columns like top rows, bottom rows, or data within an index range play a significant role.

Following are the functions that help in selecting the subset of the DataFrame:

Attribute	Description
`DataFrame.head(n)`	**It is used to select top 'n' rows in DataFrame.**
`DataFrame.tail(n)`	**It is used to select bottom 'n' rows in DataFrame.**
`DataFrame.at`	**It is used to get and set the particular value of DataFrame using row and column labels.**
`DataFrame.iat`	**It is used to get and set the particular value of DataFrame using row and column index positions.**
`DataFrame.get(key)`	**It is used to get the value of a key in DataFrame where Key is the column name.**
`DataFrame.loc()`	**It is used to select a group of data based on the row and column labels. It is used for slicing and filtering of the DataFrame.**
`DataFrame.iloc()`	**It is used to select a group of data based on the row and column index position. Use it for slicing and filtering the DataFrame.**

In [ ]:
```python
# Create DataFrame from dict
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# select top 2 rows
student_df.head(2)
```

In [ ]:
```python
# select bottom 2 rows
student_df.tail(2)
```

In [ ]:
```python
# select value at row index 0 and column 'Name'
student_df.at[0, 'Name']
```

In [ ]:
```python
# select value at first row and first column
student_df.iat[0, 0]
```

In [ ]:
```python
# select values of 'Name' column
student_df.get('Name')
```

In [ ]:
```python
# select values from row index 0 to 2 and 'Name' column
student_df.loc[0:2, ['Name']]
```

In [ ]:
```python
# select values from row index 0 to 2(exclusive) and column position 0 to 2(exclusive)
student_df.iloc[0:2, 0:2]
```

DataFrame modification

DataFrame is similar to any excel sheet or a database table where we need to insert new data or drop columns and rows if not required. Such data manipulation operations are very common on a DataFrame.

In this section, we discuss the data manipulation functions of the DataFrame.

Insert columns

Sometimes it is required to add a new column in the DataFrame. `DataFrame.insert()` function is used to insert a new column in DataFrame at the specified position.

In the below example, we insert a new column **'Class'** as a third new column in the DataFrame with default value **'A'** using the syntax:

`df.insert(loc = col_position, column = new_col_name, value = default_value)`

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# insert new column in dataframe and display
student_df.insert(loc=2, column="Class", value='A')
student_df
```

Drop columns

DataFrame may contain redundant data, in such cases, we may need to delete such data that is not required. `DataFrame.drop()` function is used to **delete the columns from DataFrame**.

In the below example, we delete the **"Age"** column from the student DataFrame using `df.drop(columns=[col1,col2...])`.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# delete column from dataframe
student_df = student_df.drop(columns='Age')
student_df
```

Apply condition

We may need to update the value in the DataFrame based on some condition. `DataFrame.where()` function is used to replace the value of DataFrame, where the condition is `False`.

Syntax:

`where(filter, other=new_value)`

It applies the filter condition on all the rows in the DataFrame, as follows:

- If the filter condition returns `False`, then it updates the row with the value specified in `other` parameter.
- If the filter condition returns `True`, then it does not update the row.

In the below example, we want to replace the student marks with '0' where marks are less than 80. We pass a filter condition `df['Marks'] > 80` to the function.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# Define filter condition
filter = student_df['Marks'] > 80
student_df['Marks'].where(filter, other=0, inplace=True)
student_df
```

DataFrame filter columns

Datasets contain massive data that need to be analyzed. But, sometimes, we may want to analyze relevant data and filter out all the other data. In such a case, we can use `DataFrame.filter()` function to fetch only required data from DataFrame.

It returns the subset of the DataFrame by applying conditions on each row index or column label as specified using the below syntax.

Syntax:

`df.filter(like = filter_cond, axis = 'columns' or 'index')`

It applies the condition on each row index or column label.

- If the condition passed then, it includes that row or column in the resultant DataFrame.
- If the condition failed, then it does not have that row or column in the resulting DataFrame.

> **Note:** It applies the filter on row index or column label, not on actual data.

In the below example, we only include the column with a column label that starts with **'N'**.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# apply filter on dataframe
student_df = student_df.filter(like='N', axis='columns')
student_df
```

DataFrame rename columns

While working with DataFrame, we may need to **rename the column** or row index. We can use `DataFrame.rename()` function to alter the row or column labels.

We need to pass a dictionary of key-value pairs as input to the function. Where key of the `dict` is the existing column label, and the value of `dict` is the new column label.

`df.rename(columns = {'old':'new'})`

It can be used to rename single or multiple columns and row labels.

In the below example, we rename column **'Marks'** to **'Percentage'** in the student DataFrame.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# rename column
student_df = student_df.rename(columns={'Marks': 'Percentage'})
student_df
```

DataFrame Join & Merge

In most of the use cases of Data Analytics, data gathered from multiple sources, and we need to combine that data for further analysis. In such instances, join and merge operations are required.

`DataFrame.join()` function is used to join one DataFrame with another DataFrame as `df1.join(df2)`

In the below example, we joined two different DataFrames to create a new resultant DataFrame.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat'], 'Age': [20, 21]}
student_df = pd.DataFrame(student_dict)
student_df

marks_dict = {'Marks': [85.10, 77.80]}
marks_df = pd.DataFrame(marks_dict)
marks_df
```

In [ ]:
```python
joined_df = student_df.join(marks_df)
joined_df
```

`pandas.merge()` function is used to update the content of one DataFrame with the content from another DataFrame:

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Davis'], 'Age': [20, 21, 23]}
student_df_part1 = pd.DataFrame(student_dict)
student_df_part1
```

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat','Alice'], 'Marks': [85.10, 77.80, 90.00]}
student_df_part2 = pd.DataFrame(student_dict)
student_df_part2
```

In [ ]:
```python
pd.merge(student_df_part1, student_df_part2, how='left', on='Name')
```

In [ ]:
```python
pd.merge(student_df_part1, student_df_part2, how='right', on='Name')
```

In [ ]:
```python
pd.merge(student_df_part1, student_df_part2, how='inner', on='Name')
```

DataFrame GroupBy

`GroupBy` operation means splitting the data and then combining them based on some condition. Large data can be divided into logical groups to analyze it.

`DataFrame.groupby()` function groups the DataFrame row-wise or column-wise based on the condition.

If we want to analyze each class's average marks, we need to combine the student data based on the 'Class' column and calculate its average using `df.groupby(col_label).mean()` as shown in the below example.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Class': ['A', 'B', 'A'], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# apply group by
student_df = student_df.groupby('Class').mean()
student_df
```

In [ ]:
```python
student_df.reset_index()
```

DataFrame Iteration

DataFrame iteration means visiting each element in the DataFrame one by one. While analyzing a DataFrame, we may need to iterate over each row of the DataFrame.

There are multiple ways to iterate a DataFrame. We will see the function `DataFrame.iterrows()`, which can loop a DataFrame row-wise. It returns the index and row of the DataFrame in each iteration of the for a loop.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85, 77]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# Iterate all the rows of DataFrame
for index, row in student_df.iterrows():
    print(index, row, "\n")
```

DataFrame Sorting

Data Analyst always needs to perform different operations on the underlying data like merge, sort, concatenate, etc. The most frequently used operation is the sorting of data. Sorted data becomes easy to analyze and inferred.

The `DataFrame.sort_values()` function is used to sort the DataFrame using one or more columns in ascending (default) or descending order.

In the below example, we sort the student data based on the **'Marks'**.

In [ ]:
```python
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19], 'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)
student_df
```

In [ ]:
```python
# rename column
student_df = student_df.sort_values(by=['Marks'])
student_df
```

DataFrame conversion

After all the processing on DataFrame, we will get the expected data in the DataFrame. But, we may require to convert the DataFrame back to its original formats like CSV file or `dict`, or we may need to convert it to another format for further action like storing it into the Database as SQL table format.

Pandas have provided plenty of functions to convert the DataFrames into many different formats.

For example, `DataFrame.to_dict()` function is used to converts the **DataFrame into a Python dictionary object.

Below is the example of a DataFrame which we need to convert into the Python `dict`.

Let's see how we can use `DataFrame.to_dict()` function to convert the DataFrame into the Python dictionary. By default, it creates the dictionary with keys as column labels and values as mapping of the row index and data.

In [ ]:
```python
# convert dataframe to dict
dict = student_df.to_dict()
print(dict)
```

In [ ]:
```python
```